ΟΙΚΟΝΟΜΙΚΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΑΘΗΝΩΝ
ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS
ΣΧΟΛΗ ΔΙΟΙΚΗΣΗΣ ΕΠΙΧΕΙΡΗΣΕΩΝ
SCHOOL OF BUSINESS
ΤΜΗΜΑ ΔΙΟΙΚΗΤΙΚΗΣ ΕΠΙΣΤΗΜΗΣ & ΤΕΧΝΟΛΟΓΙΑΣ
DEPARTMENT OF MANAGEMENT SCIENCE & TECHNOLOGY

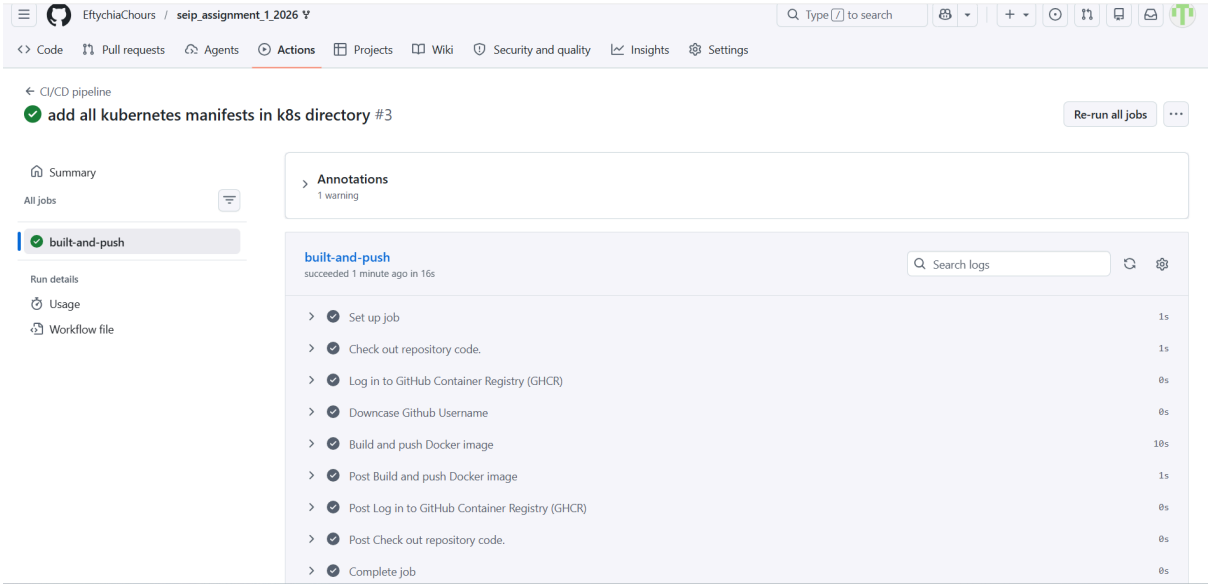
Software Engineering in Practice — Assignment 1 (2026)

Course: Software Engineering (Lab)
Assignment: Advanced DevOps: Production-Grade CI/CD, External Configuration, and Orchestration
Student Name: Eftychia Choursoglou
Student ID / AM: t8230164
Date: May 2026

1. GitHub Repository Link

Public Repository URL:
https://github.com/EftychiaChours/seip_assignment_1_2026

2. CI/CD Proof:



3. Cluster State Proof:

output of:

- kubectl get all -n default

```
PS C:\Users\eutux\seip_assignment_1_2026> kubectl get all -n default
NAME                                       READY   STATUS    RESTARTS   AGE
pod/echo-api-deployment-759b88cbc6-2gn6f   1/1     Running   0          89s
pod/echo-api-deployment-759b88cbc6-rlvxg   1/1     Running   0          89s
pod/echo-api-deployment-759b88cbc6-x5p4w   1/1     Running   0          89s

NAME                       TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
service/echo-api-service   ClusterIP   10.101.243.129   <none>        80/TCP    89s
service/kubernetes         ClusterIP   10.96.0.1        <none>        443/TCP   4m6s

NAME                                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/echo-api-deployment   3/3     3            3           89s

NAME                                             DESIRED   CURRENT   READY   AGE
replicaset.apps/echo-api-deployment-759b88cbc6   3         3         3       89s
```

- kubectl get configmap,secret

```
PS C:\Users\eutux\seip_assignment_1_2026> kubectl get configmap,secret
NAME                            DATA   AGE
configmap/echo-api-config       2      18m
configmap/kube-root-ca.crt      1      27m

NAME                         TYPE     DATA   AGE
secret/echo-api-secret       Opaque   1      18m
```

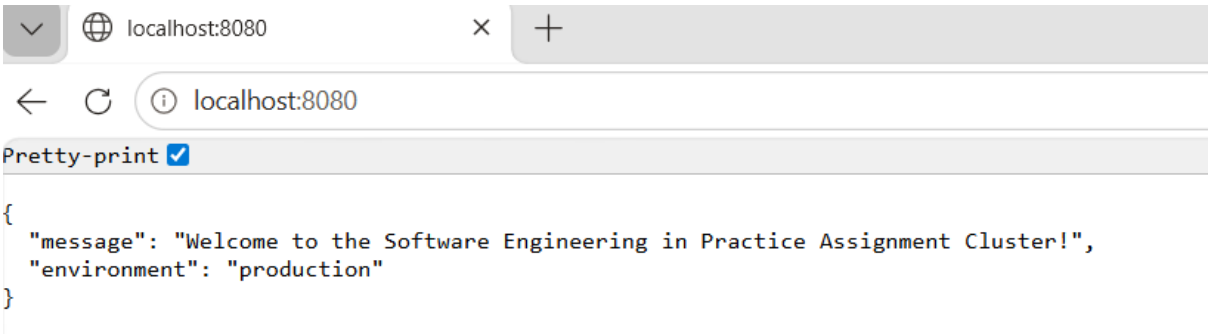
4. Application Verification Proof:

Command:

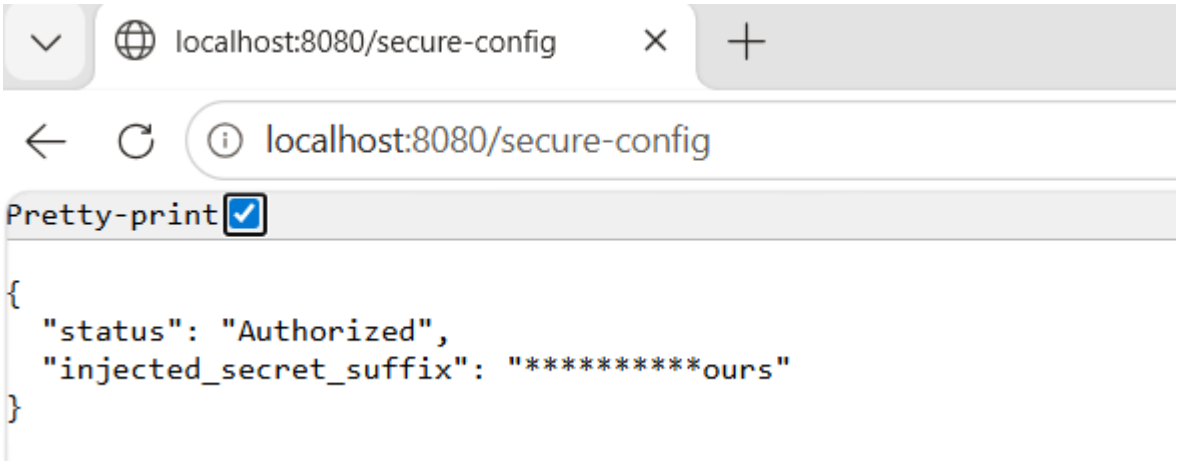
```
PS C:\Users\eutux\seip_assignment_1_2026> kubectl port-forward --address localhost service/echo-api-service 8080:80
Forwarding from 127.0.0.1:8080 -> 3000
Forwarding from [::1]:8080 -> 3000
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

browser engine :

- http://localhost:8080/



- http://localhost:8080/secure-config



5. AI Reflection & Future Outlook:

- **AI Integration:** In this assignment, Gemini was utilized as an interactive educational facilitator and technical collaborator. The AI helped me understand the assignment requirements better and it mainly helped me figure out how to use each tool (Docker, GitHub Actions, Kubernetes) and get familiar with them.
- **Utility Analysis:** The aspects of the AI assistance that I found most useful were:

1. Understanding Kubernetes Structures: Gemini helped me understand how the YAML manifests (deployment.yaml, service.yaml, etc.) are structured and how they interact with each other.
2. Explaining Technical Concepts: It provided simple explanations for complex mechanics, such as why data in the Secret file needs to be Base64 encoded.
3. Explaining Cluster Behavior: It helped me understand what was happening under the hood, like explaining why an old ReplicaSet shows 0 replicas after a configuration change.

- **Friction Points:** The main friction points during the assignment were local environment bugs and network issues where the AI tools could not look at my screen directly to see the exact context:
    1. The Image Name Error: At first, my pods were stuck on InvalidImageName because I had left a wrong character inside the image link in deployment.yaml. Gemini helped me notice that the error was just a typo in the name and guided me to fix it.
    2. Local Network and Port-Forward Freezes: When I tried to run kubectl port-forward, the connection kept freezing and throwing timeouts, meaning the web pages wouldn't load. Gemini gave me the manual steps to fix the loopback binding (--address localhost).
    3. The Second Link Issue: While the first link (http://localhost:8080/) loaded successfully, the second link (/secure-config) refused to connect no matter what we tried.
    4. Manual Resolution: Since the AI steps for network debugging couldn't bypass the local Docker Desktop freeze that day, I decided to stop. The

next day, I manually restarted everything, did a full reset (minikube delete and a fresh minikube start) and re-applied all the manifests from scratch. This completely cleared the stuck network bridges, and both links worked perfectly.

- **Future Architectural Outlook:** If I had an extra week or needed to scale this project for a real-world enterprise system, I would implement the following production upgrades:
    1. Replace Port-Forwarding with an Ingress Controller: Port-forwarding is only for local testing. For a real production system, I would deploy an Ingress Controller to guide real internet traffic to the application using a proper domain name and secure HTTPS encryption.
    2. Automate Deployments with GitOps (ArgoCD): Instead of running kubectl apply manually from my terminal, I would use ArgoCD. In this way, the process would be automated by syncing the Kubernetes cluster directly with the GitHub repository whenever a change would be made.
    3. Add Prometheus Monitoring: To ensure the application is running healthy 24/7, I would add Prometheus. That is really important, as it monitors resource usage (CPU/RAM) and alerts the team immediately if a pod crashes or gets overloaded.
    4. Integrate Image Scanning for Security: I would add a security scanner tool inside the GitHub Actions pipeline to automatically check the Docker images for vulnerabilities or leaks before pushing them to the registry.